Manuel de Conception

Compilateur Deca - Projet Génie Logiciel

Équipe GL25

Nathan Nicolas Alexandre Iles Noah

Janvier 2026

Résumé

Ce document présente l'architecture du compilateur Deca développé dans le cadre du projet Génie Logiciel de l'Ensimag. Il décrit les différents composants du projet, leurs dépendances, la définition des incréments de développement et la méthode de test adoptée.

Table des matières

1 Introduction **4**

2 Principes de conception **4**

3 Vue d'ensemble de l'architecture **4**
3.1 Liste des composants 5
3.2 Diagramme de dépendances 6

4 Spécification des composants **6**
4.1 Package `fr.ensimag.deca` – Point d'entrée 6
4.2 Package `fr.ensimag.deca.syntax` – Analyse lexicale et syntaxique 7
4.3 Package `fr.ensimag.deca.context` – Système de types 7
4.3.1 Hiérarchie des définitions 8
4.3.2 Environnements et erreurs 8
4.4 Package `fr.ensimag.deca.codegen` – Génération de code 8
4.5 Package `fr.ensimag.deca.tree` – Arbre syntaxique abstrait 9
4.5.1 Classes de base 9
4.5.2 Correspondance avec la grammaire 9
4.6 Fonctionnalités de l'AST : PrettyPrint et Decompile 9
4.7 Package `fr.ensimag.deca.tools` – Utilitaires 10
4.8 Package `fr.ensimag.deca.ir` – Représentation Intermédiaire (IR) 10
4.9 Package `fr.ensimag.ima.pseudocode` – Structures IMA (Backend) 10
4.10 Package `fr.ensimag.ima.pseudocode.instructions` – Instructions IMA 10
4.11 Packages Backend RISC-V 11
4.11.1 Package `fr.ensimag.riscv.pseudocode` 11
4.11.2 Package `fr.ensimag.deca.backend.riscv.env` 11
4.11.3 Package `fr.ensimag.deca.backend.riscv` 12
4.11.4 Flux de génération 12

5 Choix architecturaux et justifications **12**
5.1 Intégration de l'extension IR / RISC-V 12
5.1.1 Fonctionnement en parallèle 12
5.1.2 Justification 12
5.2 Architecture interne : Pattern Manager 13
5.2.1 Choix 13
5.2.2 Justification 13
5.3 Gestion des portées – `EnvironmentExp` 13
5.3.1 Structure de données 13
5.3.2 Algorithme de résolution 13
5.3.3 Justification 13
5.4 Allocation de registres – `RegisterManager` 14
5.4.1 Modèle à deux niveaux 14

5.4.2 Algorithme d'allocation . . . 14
5.4.3 Gestion du débordement . . . 14
5.4.4 Justification . . . 14
5.5 Gestion de la pile – `StackManager` . . . 14
5.5.1 Rôle . . . 14
5.5.2 Comptabilité . . . 15
5.5.3 Vérification runtime . . . 15
5.5.4 Justification . . . 15
5.6 Gestion des variables – `VarManager` et Rematérialisation . . . 15
5.6.1 Principe de rematérialisation . . . 15
5.6.2 Cycle de vie . . . 15
5.6.3 Justification . . . 15

6 Optimisations – `OptimizationsManager` 16
6.1 Multiplication par puissance de 2 . . . 16
6.2 Division entière par puissance de 2 . . . 16
6.3 Modulo par puissance de 2 . . . 16
6.4 Division flottante par 2 . . . 16

7 Support Objet et Tables de Méthodes Virtuelles 17
7.1 Organisation mémoire des objets . . . 17
7.2 Structure des VTables . . . 17
7.2.1 Justification du slot 0 . . . 17
7.3 Algorithme d'attribution des index de méthodes . . . 17
7.3.1 Exemple . . . 18
7.4 Génération des VTables – `StaticManager` . . . 18
7.4.1 Passe 1 : Attribution des adresses . . . 18
7.4.2 Passe 2 : Génération du code . . . 18
7.5 Dispatch dynamique de méthodes . . . 18
7.6 Initialisation des objets . . . 19
7.7 Vérifications de sécurité . . . 19

8 Définition des incréments 20
8.1 Justification de l'ordre des incréments . . . 20
8.2 Ordre de codage par package . . . 20

9 Documentation générée 21
9.1 Javadoc . . . 21
9.2 Site Maven . . . 21

10 Flux de compilation 22

11 Conclusion 22

1 Introduction

Le compilateur Deca est un compilateur pour le langage Deca, un langage orienté objet pédagogique et ressemblant à Java. Il compile des fichiers sources `.deca` vers du code assembleur IMA, exécutable sur une machine virtuelle.

Le projet utilise notamment Maven pour le système de build, et ANTLR4 pour l'analyse lexicale et syntaxique.

2 Principes de conception

L'architecture du compilateur repose sur plusieurs principes d'ingénierie logicielle qui guident l'ensemble de nos choix techniques :

- **Séparation stricte des responsabilités** : Chaque phase de compilation (analyse, vérification, génération) est isolée dans un package dédié, conformément au découpage A/B/C spécifié dans le sujet.
- **Une classe = une responsabilité** : Les managers du package `codegen` illustrent ce principe : `RegisterManager` gère uniquement l'allocation de registres, `StackManager` uniquement la pile, etc.
- **Découplage frontend/backend** : L'introduction d'une représentation intermédiaire (IR) permet d'ajouter de nouvelles cibles (RISC-V) sans modifier le frontend, et inversement.
- **Conformité aux spécifications** : L'AST suit fidèlement la grammaire de *SyntaxeAbstraite*, les environnements respectent les règles de *SyntaxeContextuelle*, et la génération de code est conforme aux conventions IMA.
- **Extensibilité anticipée** : Le pattern Manager et l'architecture en couches facilitent l'ajout d'optimisations ou de nouvelles fonctionnalités sans refonte majeure.

3 Vue d'ensemble de l'architecture

Le compilateur est structuré en trois couches principales :

- **Le Frontend** (Package `fr.ensimag.deca` et ses sous-packages) : Il regroupe l'analyse du code source. Il contient notamment :
  - `syntax` : Analyse lexicale et syntaxique.
  - `tree` : Structure de l'Arbre Syntaxique Abstrait (AST).
  - `context` : Vérification de types et sémantique.
  - `codegen` : Génération de code vers la représentation intermédiaire ou directement vers ima.
- **La Représentation Intermédiaire** (Package `fr.ensimag.deca.ir`) : Structure pivot découplant l'analyse du langage de la machine cible.
- **Les Backends** (Packages `fr.ensimag.deca.ima` et `riscv`) : Responsables de la traduction de l'IR vers l'assembleur spécifique (IMA ou RISC-V).

3.1 Liste des composants

Package	Classes	Rôle
Frontend (Deca)		
`fr.ensimag.deca`	5	Point d'entrée, CLI
`fr.ensimag.deca.syntax`	10	Analyse lexicale et syntaxique
`fr.ensimag.deca.context`	20	Analyse contextuelle / système de types
`fr.ensimag.deca.codegen`	10	Génération de code
`fr.ensimag.deca.tools`	3	Outils et utilitaires
`fr.ensimag.deca.tree`	86	Arbre syntaxique abstrait (AST)
Représentation intermédiaire (IR)		
`fr.ensimag.deca.ir`	4	Représentation intermédiaire
`fr.ensimag.deca.ir.control`	1	Contrôle d'erreurs IR
`fr.ensimag.deca.ir.display`	2	Affichage / pretty-printing IR
`fr.ensimag.deca.ir.instructions`	14	Instructions IR
`fr.ensimag.deca.ir.operands`	15	Opérandes IR
Backend (générateurs)		
`fr.ensimag.deca.backend.ima`	1	Génération IMA
`fr.ensimag.deca.backend.qbe`	1	Génération QBE
`fr.ensimag.deca.backend.riscv`	2	Génération RISC-V
`fr.ensimag.deca.backend.riscv.env`	6	Environnement RISC-V
Pseudocode IMA		
`fr.ensimag.ima.pseudocode`	26	Structures assembleur IMA
`fr.ensimag.ima.pseudocode.instructions`	54	Instructions IMA
Pseudocode RISC-V		
`fr.ensimag.riscv.pseudocode`	3	Structures assembleur RISC-V
Total	**263**	

Table 1 – Répartition des classes par package

3.2 Diagramme de dépendances

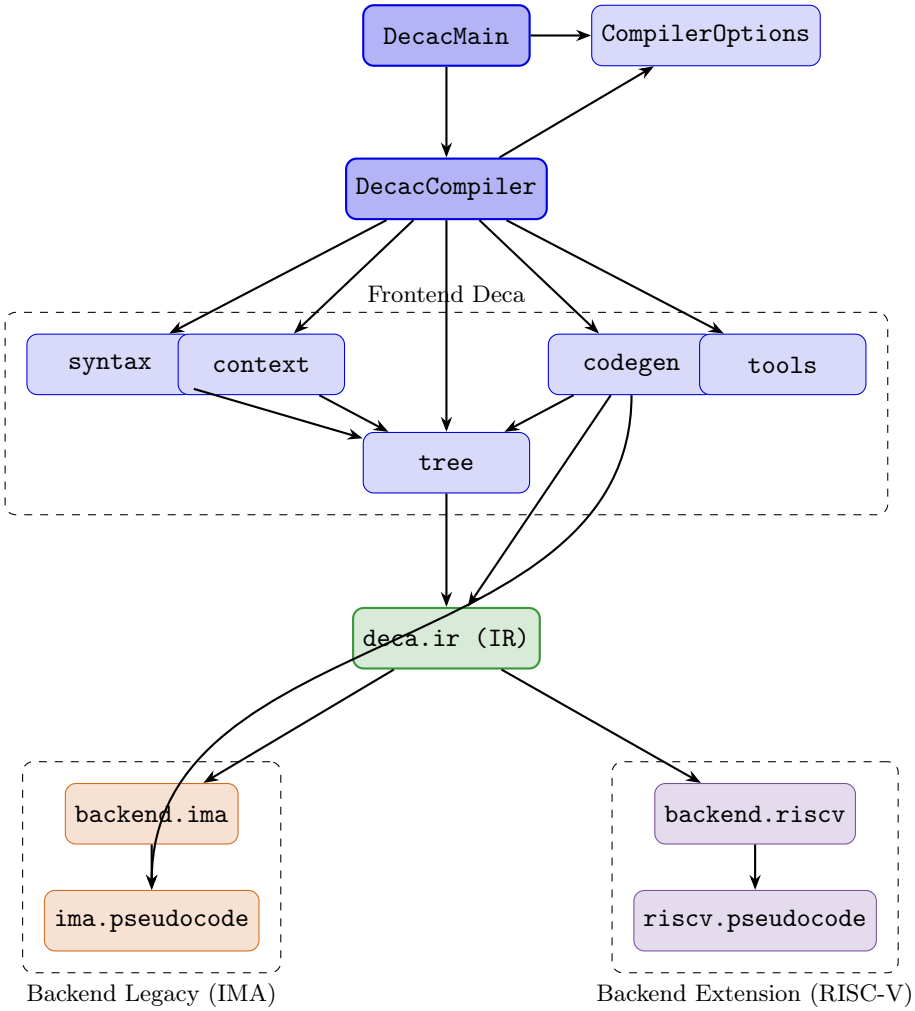


FIGURE 1 – Diagramme de dépendances : Pipeline Legacy (codegen → ima.pseudocode) et Extension (IR → backends)

4 Spécification des composants

4.1 Package `fr.ensimag.deca` – Point d'entrée

Ce package contient le point d'entrée du compilateur et la gestion des options en ligne de commande.

Classe	Description
`DecacMain`	Point d'entrée `main()`, gère la compilation parallèle de plusieurs fichiers. Appelle les phases A, B, C dans l'ordre
`DecacCompiler`	Orchestrateur principal de compilation, une instance par fichier source
`CompilerOptions`	Gestion des options CLI (`-r`, `-p`, `-v`, `-b`, `-n`, niveaux debug/optimisation)
`CLIException`	Exception pour les erreurs d'arguments en ligne de commande
`DecacFatalError`	Exception pour les erreurs fatales irrécupérables

TABLE 2 – Classes du package `fr.ensimag.deca`

4.2 Package `fr.ensimag.deca.syntax` – Analyse lexicale et syntaxique

Ce package gère l'analyse lexicale (tokenisation) et syntaxique (parsing) du code source Deca. Il utilise ANTLR4 pour générer le lexer et le parser à partir des grammaires `DecaLexer.g4` et `DecaParser.g4`.

N.B. : Il n'y a pas d'erreur lexicale à proprement parler car le processus va toujours jusqu'au parser. Les erreurs détectées au niveau du lexer sont remontées comme des erreurs de parsing.

Classe	Description
`AbstractDecaLexer`	Classe de base pour le lexer (générée par ANTLR)
`AbstractDecaParser`	Classe de base pour le parser (générée par ANTLR)
`SyntaxManager`	Utilitaires : `unescape()`, `bound()`, `getInt()`, `getFloat()`
`DecaRecognitionException`	Classe de base pour les exceptions de reconnaissance
`LexerError`	Erreur lexicale (token non reconnu, overflow littéral). Étend `DecaRecognitionException`
`ParserError`	Erreur syntaxique (structure invalide). Étend `DecaRecognitionException`
`DecacErrorListner`	Listener personnalisé pour capturer les erreurs ANTLR
`CircularInclude`	Erreur d'inclusion circulaire (`#include`)
`IncludeFileNotFound`	Erreur fichier `#include` non trouvé
`InvalidLValue`	Erreur L-value invalide (ex : `5 = x`)

TABLE 3 – Classes du package `fr.ensimag.deca.syntax`

Note sur les erreurs : `LexerError` et `ParserError` étendent tous les deux `DecaRecognitionException`. Au final, ce sont des erreurs de parsing, mais on est capable de distinguer si l'erreur provient du lexer ou du parser.

4.3 Package `fr.ensimag.deca.context` – Système de types

Ce package implémente le système de types et les vérifications contextuelles (sémantiques) du langage Deca, conformément aux règles définies dans *SyntaxeContextuelle* du poly-projet. Les environnements (`EnvironmentType`, `EnvironmentExp`) respectent la structure de portées imbriquées spécifiée, et les règles de typage (sous-typage, conversions implicites int→float) sont fidèlement implémentées.

4.3.1 Hiérarchie des définitions

Classe	Description
`Definition`	Classe abstraite de base pour les définitions
`TypeDefinition`	Définition d'un type
`ClassDefinition`	Définition d'une classe (champs, méthodes)
`ExpDefinition`	Base pour variable/paramètre/méthode/champ
`VariableDefinition`	Définition d'une variable locale ou globale
`ParamDefinition`	Définition d'un paramètre de méthode
`FieldDefinition`	Définition d'un champ de classe
`MethodDefinition`	Définition d'une méthode
`Signature`	Signature de méthode (type retour + types paramètres)

TABLE 4 – Hiérarchie des définitions

4.3.2 Environnements et erreurs

Classe	Description
`EnvironmentType`	Dictionnaire des types déclarés (types primitifs + classes)
`EnvironmentExp`	Environnement chaîné pour les portées imbriquées (blocs, classes)
`ContextualError`	Exception levée lors d'une erreur contextuelle (type incompatible, variable non déclarée, etc.)

TABLE 5 – Classes d'environnement

4.4 Package `fr.ensimag.deca.codegen` – Génération de code

Ce package contient les managers responsables de la génération de code assembleur.

Classe	Description
`GenerationContext`	Agrège tous les managers, contexte principal de génération
`RegisterManager`	Allocation et gestion des registres R0-R15
`StackManager`	Gestion PUSH/POP et analyse statique de la pile
`LabelManager`	Génération de labels uniques (boucles, conditions)
`ErrorManager`	Génération des labels et code d'erreur runtime
`VarManager`	Gestion des variables locales et rematérialisation
`OptimizationsManager`	Gestion des optimisations (puissance de 2, etc.)
`StaticManager`	Gestion des VTables et allocation mémoire statique
`ReservedVar`	Représentation d'une variable réservée dans un registre
`GenerationError`	Exception/Warning levée pendant la génération de code

TABLE 6 – Classes du package `fr.ensimag.deca.codegen`

Note sur `GenerationError` : Cette classe est utilisée dans le but d'optimiser le compilateur. Elle permet d'émettre des *Warnings* et d'anticiper les erreurs à l'exécution. Quand elle est levée, on pourrait déjà générer des erreurs potentielles. Elle est prévue pour une pipeline future d'optimisation.

4.5 Package `fr.ensimag.deca.tree` – Arbre syntaxique abstrait

Ce package contient l'implémentation de l'AST (Abstract Syntax Tree) avec 86 classes. Il utilise le pattern Interpreter : chaque nœud de l'arbre implémente les méthodes `verifyXXX()` et `codeGenXXX()`.

Note : Le package `tree` est utilisé par les trois phases de compilation (A, B, C), car l'AST est la structure centrale qui traverse toutes les étapes.

4.5.1 Classes de base

Classe	Description
`Tree`	Classe de base de toute la hiérarchie, encapsule `Location`
`AbstractProgram`	Point d'entrée de l'AST (`verifyProgram()`, `codeGenProgram()`)
`AbstractMain`	Programme principal (`main`)
`AbstractInst`	Classe de base pour les instructions
`AbstractExpr`	Classe de base pour les expressions
`AbstractBinaryExpr`	Expression binaire (deux opérandes)
`AbstractUnaryExpr`	Expression unaire (un opérande)
`AbstractLValue`	L-value (utilisable à gauche d'une assignation)

TABLE 7 – Classes de base de l'AST

4.5.2 Correspondance avec la grammaire

L'architecture des classes de l'AST suit fidèlement la spécification *SyntaxeAbstraite* du poly-projet. Chaque règle de production grammaticale est implémentée par une classe Java dédiée :

— Les non-terminaux abstraits (`inst`, `expr`, `decl_var`) correspondent aux classes `Abstract*`

— Les productions concrètes (`while`, `plus`, `decl_var`) correspondent aux classes concrètes (`While`, `Plus`, `DeclVar`)

Cette correspondance directe facilite la vérification de conformité et la maintenance. La liste exhaustive des 86 classes est disponible via la Javadoc générée.

4.6 Fonctionnalités de l'AST : PrettyPrint et Decompile

L'AST supporte deux modes d'affichage pour le débogage et la vérification :

— **PrettyPrint** : Affichage formaté de l'arbre syntaxique avec indentation, permettant de visualiser la structure hiérarchique de l'AST. Utile pour le débogage des passes de compilation.

— **Decompile** : Régénération du code source Deca à partir de l'AST. Permet de vérifier que le parsing et la construction de l'AST sont corrects en comparant le code décompilé avec le code source original.

Ces fonctionnalités sont accessibles via les options `-p` (parse only + decompile) du compilateur.

4.7 Package `fr.ensimag.deca.tools` – Utilitaires

Classe	Description
`SymbolTable`	Table de symboles garantissant l'unicité des identificateurs
`IndentPrintStream`	PrintStream avec gestion automatique de l'indentation
`DecacInternalError`	Exception pour les bugs internes du compilateur

TABLE 8 – Classes du package `fr.ensimag.deca.tools`

4.8 Package `fr.ensimag.deca.ir` – Représentation Intermédiaire (IR)

Ce package définit la représentation intermédiaire commune utilisée entre le frontend Deca et les différents backends (IMA, RISC-V). L'IR permet de découpler la génération de code spécifique à chaque architecture.

Sous-package	Description
`ir` (racine)	Classes principales de l'IR (IRProgram, etc.)
`ir.control`	Gestion des erreurs IR (IRError)
`ir.operands`	Opérandes IR (IRLabel, IRVariable, IRData, IRScratch, etc.)
`ir.display`	Affichage et pretty-printing de l'IR
`ir.instructions`	Instructions IR (sauts, opérations, etc.)

TABLE 9 – Structure du package IR

4.9 Package `fr.ensimag.ima.pseudocode` – Structures IMA (Backend)

Ce package définit les structures de données pour représenter un programme assembleur IMA.

Classe	Description
`IMAProgram`	Programme IMA complet (liste de lignes)
`AbstractLine`, `Line`	Ligne = label + instruction + commentaire
`Instruction`	Classe abstraite de base pour les instructions
`Register`, `GPRegister`	Registres généraux R0-R15
`Label`, `LabelOperand`	Étiquettes pour les branchements
`Operand`, `DVal`, `DAddr`	Opérandes (valeurs, adresses)
`ImmediateInteger/Float/String`	Valeurs immédiates
`RegisterOffset`	Adressage indirect avec offset

TABLE 10 – Classes principales du package `fr.ensimag.ima.pseudocode`

4.10 Package `fr.ensimag.ima.pseudocode.instructions` – Instructions IMA

Ce package contient l'implémentation objet du jeu d'instructions de l'architecture IMA, conformément à la documentation *IMA - Guide de l'utilisateur* fournie avec le projet.

Nous avons fait le choix de représenter chaque opcode par une classe Java dédiée (ex : `ADD`, `LOAD`, `BSR`). Ce package contient 54 classes couvrant l'intégralité des catégories définies dans la

spécification IMA (Arithmétique, Déplacement, Branchements, Pile, etc.), permettant une génération de code fortement typée et polymorphe. Les conventions d'appel (sauvegarde de registres, passage de paramètres via la pile) respectent les recommandations du poly.

4.11 Packages Backend RISC-V

Ces packages fournissent un backend alternatif pour générer du code assembleur RISC-V à partir de la représentation intermédiaire. L'architecture suit une séparation claire des responsabilités via plusieurs design patterns (Visitor, Facade, Value Object).

Package	Description
`fr.ensimag.deca.backend.riscv`	Orchestration de la génération RISC-V et visiteur IR
`fr.ensimag.deca.backend.riscv.env`	Managers d'environnement (pile, données, erreurs, formatage)
`fr.ensimag.riscv.pseudocode`	Représentation abstraite du programme assembleur

TABLE 11 – Packages du backend RISC-V

4.11.1 Package `fr.ensimag.riscv.pseudocode`

Ce package fournit les structures de données pour représenter un programme assembleur RISC-V.

- **RVProgram** : Représentation complète d'un programme avec sections `.data` et `.text`. Gère l'ajout de lignes et la génération du fichier assembleur final.
- **AbstractLine** : Classe abstraite base pour toute ligne du programme (Template Method Pattern).
- **Line** : Implémentation concrète d'une ligne simple avec contenu immuable.

4.11.2 Package `fr.ensimag.deca.backend.riscv.env`

Ce package contient les managers qui gèrent l'environnement de génération. `RVContext` agit comme façade centrale donnant accès à tous les composants.

- **RVContext** : Point d'accès unique aux managers, agrège les références au générateur, IR et programme de sortie.
- **RVStackManager** : Calcule le plan d'adressage de la pile (variables et scratches) avec alignement à 16 bytes.
- **RVOffsets** : Value Object immuable stockant les mappings variable/scratch → offset.
- **RVDataManager** : Gère les mouvements de données entre mémoire et registres (`li`, `lw/flw`, `sw/fsw`, conversions float).
- **RVFormatter** : Formate l'assembleur avec indentation cohérente, gestion des labels, échappement des chaînes.
- **RVErrorManager** : Génère les vérifications runtime (division par zéro, overflow arithmétique, floats invalides) avec branchements vers handlers d'erreurs.

4.11.3 Package `fr.ensimag.deca.backend.riscv`

Ce package contient l'orchestrateur principal et le visiteur qui transforme l'IR en assembleur.

— **RVGenerator** : Orchestre la génération en 4 phases : initialisation du contexte, émission des données, génération du code via visiteur, émission des handlers d'erreurs.

— **RVVisitor** : Implémente `IRVisitor` pour transformer chaque instruction IR en assembleur RISC-V correspondant. Supporte les affectations, opérations arithmétiques (avec vérifications overflow), entrées/sorties via syscalls, et contrôle de flux.

4.11.4 Flux de génération

1. `RVGenerator` crée le `RVContext` avec l'IR en entrée
2. Les données (`IRData`) sont émises dans la section `.data`
3. Le prologue ajuste le stack pointer (`addi sp, sp, -SIZE`)
4. `RVVisitor` visite chaque instruction IR et délègue aux managers
5. Les handlers d'erreurs runtime sont émis en fin de programme
6. `RVProgram` produit le fichier assembleur final

5 Choix architecturaux et justifications

Cette section décrit les choix de conception spécifiques à notre implémentation, leurs justifications, ainsi que les algorithmes et structures de données développés.

5.1 Intégration de l'extension IR / RISC-V

Pour ajouter le support RISC-V sans modifier le fonctionnement éprouvé de la génération IMA (rendu principal), nous avons choisi de développer l'extension en parallèle.

5.1.1 Fonctionnement en parallèle

Le compilateur dispose de deux modes de génération distincts :

1. **Mode Standard (IMA)** : Le code est généré directement via les classes du package `fr.ensimag.ima.pseudocode`, comme dans l'architecture initiale.
2. **Mode Extension (IR)** : Une nouvelle branche de code construit la Représentation Intermédiaire (`fr.ensimag.deca.ir`), qui est ensuite traduite par les backends spécifiques (RISC-V, QBE).

5.1.2 Justification

Cette séparation a été mise en place pour sécuriser le développement :

— **Stabilité** : La génération IMA reste isolée et n'est pas impactée par les développements expérimentaux de l'IR.

— **Indépendance** : L'extension peut évoluer (ajout de nouvelles instructions IR) sans nécessiter de refonte du code existant.

Les détails techniques de l'IR sont disponibles dans la **Documentation de l'Extension**.

5.2 Architecture interne : Pattern Manager

5.2.1 Choix

Le package `codegen` utilise un pattern **Manager** où un `GenerationContext` agrège plusieurs managers spécialisés (registres, pile, labels, erreurs, variables, optimisations).

5.2.2 Justification

— **Séparation des responsabilités** : Chaque manager gère un aspect précis de la génération de code, facilitant la maintenance et les tests unitaires.
— **Faible couplage** : Les managers communiquent via le contexte, évitant les dépendances circulaires.
— **Extensibilité** : Ajouter un nouveau backend (ex : RISC-V) nécessite seulement de créer de nouveaux managers sans modifier le frontend.

5.3 Gestion des portées – `EnvironmentExp`

5.3.1 Structure de données

L'environnement des expressions utilise une **liste chaînée d'environnements** où chaque niveau de portée maintient :
— Un `HashMap<Symbol, ExpDefinition>` pour les symboles locaux
— Un pointeur `parentEnvironment` vers la portée englobante

5.3.2 Algorithme de résolution

La recherche d'un symbole suit un parcours **de l'intérieur vers l'extérieur** :

1. Recherche dans le dictionnaire de la portée courante (O(1) en moyenne)
2. Si non trouvé, délégation récursive au parent
3. Retourne `null` si la racine est atteinte sans succès

Complexité : O(1) en moyenne (cas typique), O(d) au pire où d est la profondeur d'imbrication.

5.3.3 Justification

— **HashMap** : Accès O(1) grâce à l'interning des symboles via `SymbolTable`
— **Chaînage** : Implémente naturellement les règles de portée lexicale et le shadowing
— **Immutabilité du chaînage** : Le lien parent ne change jamais après construction, garantissant la cohérence

5.4 Allocation de registres – `RegisterManager`

5.4.1 Modèle à deux niveaux

Les registres R0-R15 sont partitionnés en deux catégories :

Catégorie	Plage	Usage
Scratch (temporaires)	R2 à R(max-reserved-1)	Évaluation d'expressions
Réservés (variables)	R(max-reserved) à R(max-1)	Variables locales fréquentes

TABLE 12 – Partition des registres

5.4.2 Algorithme d'allocation

Phase 1 – Réservation (au début de chaque bloc) :

1. `VarManager.reserve()` identifie les variables locales candidates
2. Calcul du nombre de registres disponibles après `-r` option
3. Attribution des registres hauts (R15, R14, ...) aux premières variables déclarées
4. Chargement initial (LOAD) des valeurs en mémoire vers les registres

Phase 2 – Utilisation (pendant la génération) :

— Les expressions utilisent les registres scratch de manière LIFO
— `canSwap(n)` vérifie si deux registres consécutifs sont disponibles
— En cas d'épuisement, débordement sur la pile via `StackManager`

5.4.3 Gestion du débordement

Quand tous les registres scratch sont utilisés :

1. Sauvegarde de l'opérande gauche sur la pile (PUSH)
2. Calcul de l'opérande droit dans le registre libéré
3. Transfert du résultat vers R0 (toujours disponible)
4. Restauration de l'opérande gauche (POP)
5. Exécution de l'opération

5.4.4 Justification

— **Registres hauts pour variables** : Évite les conflits avec les opérations binaires qui utilisent les registres bas
— **Minimum garanti (2 scratch)** : Assure que toute expression binaire peut être évaluée
— **Option -r configurable** : Permet de tester le comportement avec moins de registres

5.5 Gestion de la pile – `StackManager`

5.5.1 Rôle

Le `StackManager` est un **analyseur statique** qui trace l'utilisation de la pile pendant la compilation pour :

— Calculer la profondeur maximale de pile nécessaire
— Injecter les vérifications de débordement au runtime

5.5.2 Comptabilité

Deux compteurs sont maintenus :
— `current` : profondeur actuelle (en mots)
— `max` : profondeur maximale observée

5.5.3 Vérification runtime

À la fin de chaque génération de méthode, le `ErrorManager` injecte :

```
TSTO size    ; Test Stack Overflow
BOV error    ; Branch if Overflow
```

5.5.4 Justification

— **Analyse statique** : La profondeur de pile est déterminable à la compilation en Deca
— **Vérification anticipée** : Le test est placé à l'entrée de fonction, avant toute allocation
— **Pas de spilling dynamique** : La politique de débordement est gérée par `RegisterManager`

5.6 Gestion des variables – `VarManager` et Rematérialisation

5.6.1 Principe de rematérialisation

Les variables locales fréquemment utilisées sont maintenues en registre avec un **suivi de modification** (dirty bit) :

État	Signification
`dirty = false`	Valeur en registre = valeur en mémoire
`dirty = true`	Valeur en registre modifiée, mémoire obsolète

Table 13 – États du dirty bit

5.6.2 Cycle de vie

1. **Initialisation** : LOAD depuis la mémoire, `dirty = false`
2. **Lecture** : Accès direct au registre (pas d'accès mémoire)
3. **Écriture** : Modification du registre, `dirty = true`
4. **Flush** : STORE uniquement si `dirty = true`, puis `dirty = false`

5.6.3 Justification

— **Réduction des accès mémoire** : Les lectures sont servies depuis le registre
— **Écritures paresseuses** : Seules les variables modifiées sont écrites en mémoire

— **Priorité FIFO** : Les premières variables déclarées (souvent les plus utilisées) sont privilégiées

6 Optimisations – `OptimizationsManager`

Le `OptimizationsManager` analyse les opérations arithmétiques pour appliquer des transformations algébriques réduisant le coût en cycles processeur. Cette section détaille les algorithmes de génération utilisés pour ces cas spécifiques.

6.1 Multiplication par puissance de 2

Transformation : `x * 2^k` → k décalages à gauche (SHL)

Détection : L'expression `(n & (n-1)) == 0` pour $n > 0$ détecte les puissances de 2 en O(1).

Exemple : `x * 8` génère 3 instructions SHL au lieu d'un MUL.

6.2 Division entière par puissance de 2

Problème : Le décalage à droite (SHR) ne fonctionne correctement que pour les valeurs positives.

Solution : Génération d'un branchement conditionnel pour garantir la sémantique signée :

```
    CMP #0, Rn            ; Compare avec 0
    BLT fallback          ; Si negatif, utiliser QUO
    SHR Rn                ; Decalage rapide (k fois)
    ...
    BRA end
fallback:
    QUO diviseur, Rn      ; Division standard
end:
```

6.3 Modulo par puissance de 2

Algorithme pour valeurs positives : `r = a - ((a >> k) << k)`

Cette formule calcule le reste en trois étapes :

1. Divisant par décalage droit (partie entière)
2. Remultipliant par décalage gauche (multiple inférieur)
3. Soustrayant pour obtenir le reste

Cas négatif : Fallback vers l'instruction REM standard.

6.4 Division flottante par 2

Transformation : `x / 2.0` → `x * 0.5`

Justification : La multiplication flottante est généralement plus rapide que la division sur les processeurs modernes et l'architecture cible.

7 Support Objet et Tables de Méthodes Virtuelles

Cette section détaille l'implémentation des fonctionnalités orientées objet, en particulier le mécanisme des tables de méthodes virtuelles (VTables) qui permet le polymorphisme.

7.1 Organisation mémoire des objets

Chaque instance d'objet en mémoire (heap) suit la structure suivante :

Offset	Contenu
0	Pointeur vers la VTable de la classe
1	Premier champ (hérité ou propre)
2	Deuxième champ
...	...
n	n-ième champ

TABLE 14 – Structure d'un objet en mémoire

Taille d'allocation : `1 + nombre_de_champs` mots (instruction NEW).

7.2 Structure des VTables

Les VTables sont stockées en mémoire statique (espace GB). Chaque classe possède une VTable avec la structure :

Index	Contenu
0	Pointeur vers la VTable de la classe parente (ou null pour Object)
1	Adresse de la méthode d'index 1
2	Adresse de la méthode d'index 2
...	...
n	Adresse de la méthode d'index n

TABLE 15 – Structure d'une VTable

7.2.1 Justification du slot 0

Le pointeur vers la VTable parente permet :
— La vérification dynamique de type (`instanceof`)
— Le parcours de la hiérarchie de classes au runtime

7.3 Algorithme d'attribution des index de méthodes

L'attribution des index suit des règles strictes pour garantir la compatibilité avec l'héritage :

1. **Méthode héritée non redéfinie** : Conserve l'index du parent
2. **Méthode redéfinie (override)** : Réutilise l'index de la méthode parente
3. **Nouvelle méthode** : Obtient le prochain index disponible

Conséquence : Une méthode garde le même index dans toute la hiérarchie, permettant un dispatch en O(1).

7.3.1 Exemple

```
class Animal {
    void crier() { ... }  // index 1
    void manger() { ... } // index 2
}

class Chien extends Animal {
    void crier() { ... }  // index 1 (override)
    void aboyer() { ... } // index 3 (nouvelle)
}
```

VTable de Chien : [VTable_Animal, code.Chien.crier, code.Animal.manger, code.Chien.aboyer]

7.4 Génération des VTables – `StaticManager`

La génération se fait en deux passes :

7.4.1 Passe 1 : Attribution des adresses

1. `beginVtable()` : Initialise le compteur d'index à 1
2. Pour chaque classe (Object d'abord, puis classes utilisateur) :
   — Calcul de l'espace nécessaire : `1 + nombre_méthodes`
   — Attribution de l'index de base dans l'espace GB
3. `endVtable()` : Enregistre la taille totale

7.4.2 Passe 2 : Génération du code

Pour chaque classe :

1. Stocker le pointeur vers la VTable parente (slot 0)
2. Copier les entrées de méthodes héritées depuis la VTable parente
3. Écraser les slots des méthodes redéfinies avec les nouvelles adresses
4. Ajouter les nouvelles méthodes aux slots suivants

7.5 Dispatch dynamique de méthodes

L'appel de méthode `obj.method(args)` génère :

```
; 1. Evaluer l'objet et empiler
<code pour obj>
STORE Rn, 0(SP)      ; Sauvegarder la reference

; 2. Empiler les arguments
<code pour args>

; 3. Charger la VTable
LOAD 0(SP), Rn       ; Recuperer l'objet
```

```
LOAD 0(Rn), Rn        ; Charger ptr VTable depuis objet[0]

; 4. Appeler la methode
BSR idx(Rn)           ; Branchement indirect VTable[idx]
```

Complexité : O(1) – l'index de méthode est connu à la compilation.

7.6 Initialisation des objets

L'instruction `new ClassName()` génère :

1. **Allocation** : `NEW size, Rn` alloue sur le heap
2. **Installation VTable** :

```
LEA VTable_addr(GB), R0   ; Adresse de la VTable
STORE R0, 0(Rn)           ; Stocker dans objet[0]
```

3. **Chaînage d'initialisation** : Appel de `init.ClassName` qui appelle récursivement `init.ParentClass`

7.7 Vérifications de sécurité

Le compilateur génère des vérifications runtime :
— **Déréférencement null** : `CMP #null, Rn` + `BEQ error` avant chaque accès
— **Débordement heap** : Vérification après `NEW` via `BOV`
— **Cast invalide** : Parcours de la chaîne de VTables pour `instanceof`

8 Définition des incréments

Le développement du compilateur suit une approche incrémentale, où chaque phase réduit les risques techniques avant d'introduire de nouvelles complexités.

Phase	Nom	Description
1	Analyse lexicosyntaxique	Implémentation du lexer et parser ANTLR, construction de l'AST
2	Hello World	Premier programme compilable, génération de code minimale
3	Compilateur sans objet	Variables, expressions, instructions, fonctions (sans classes)
4	Optimisations	Amélioration des performances du code généré
5	Compilateur complet	Support complet des classes et de l'héritage

TABLE 16 – Phases de développement

8.1 Justification de l'ordre des incréments

1. **Analyse lexicosyntaxique en premier** : Permet de valider rapidement la conformité à la grammaire Deca et d'avoir un AST fonctionnel sur lequel bâtir toutes les phases suivantes.
2. **Hello World avant le reste** : Valide la chaîne complète de compilation (parsing → génération → exécution IMA) sur un cas minimal et répond aux attendus du projet.
3. **Compilateur sans objet** : Permet de stabiliser la génération de code (expressions, instructions, pile, registres) sur des cas simples. Les mécanismes fondamentaux (gestion mémoire, branchements) sont ainsi éprouvés avant l'introduction des objets.
4. **Optimisations avant les objets** : Optimiser notre compilation était un objectif important pour nous, ayant un peu d'avance nous avons décidé de nous pencher assez tôt sur cette problématique.
5. **Compilateur complet en dernier** : L'héritage, le polymorphisme et les VTables représentent la partie la plus complexe et venaient très logiquement après la partie sans objet.

8.2 Ordre de codage par package

L'ordre d'exécution est défini dans un fichier du main qui appelle les phases A, B, C séquentiellement. Chaque phase utilise des packages spécifiques :

— **Phase A (Syntax)** : `syntax` – Analyse lexicale et syntaxique, construction de l'AST
— **Phase B (Context)** : `context` – Vérifications contextuelles, typage
— **Phase C (CodeGen)** : `codegen` – Génération du code assembleur

Note importante : Le package `tree` n'est pas limité à une seule phase. Il est utilisé par les trois phases (A, B, C) car l'AST est la structure centrale traversant toute la compilation.

Phase	Package principal	Packages utilisés
A	`syntax`	`syntax`, `tree`, `tools`
B	`context`	`context`, `tree`, `tools`
C	`codegen`	`codegen`, `tree`, `ir`, `ima/riscv`

TABLE 17 – Correspondance phases/packages

9 Documentation générée

9.1 Javadoc

La documentation Javadoc est générée automatiquement à partir des commentaires du code source. Chaque package dispose d'un fichier `package-info.java` décrivant son rôle.

```
# Generation de la Javadoc
mvn javadoc:javadoc

# Consultation
xdg-open target/site/apidocs/index.html
```

9.2 Site Maven

Un site complet incluant Javadoc, rapports de qualité et métriques peut être généré :

```
# Generation du site complet
mvn site

# Consultation
xdg-open target/site/index.html
```

10 Flux de compilation

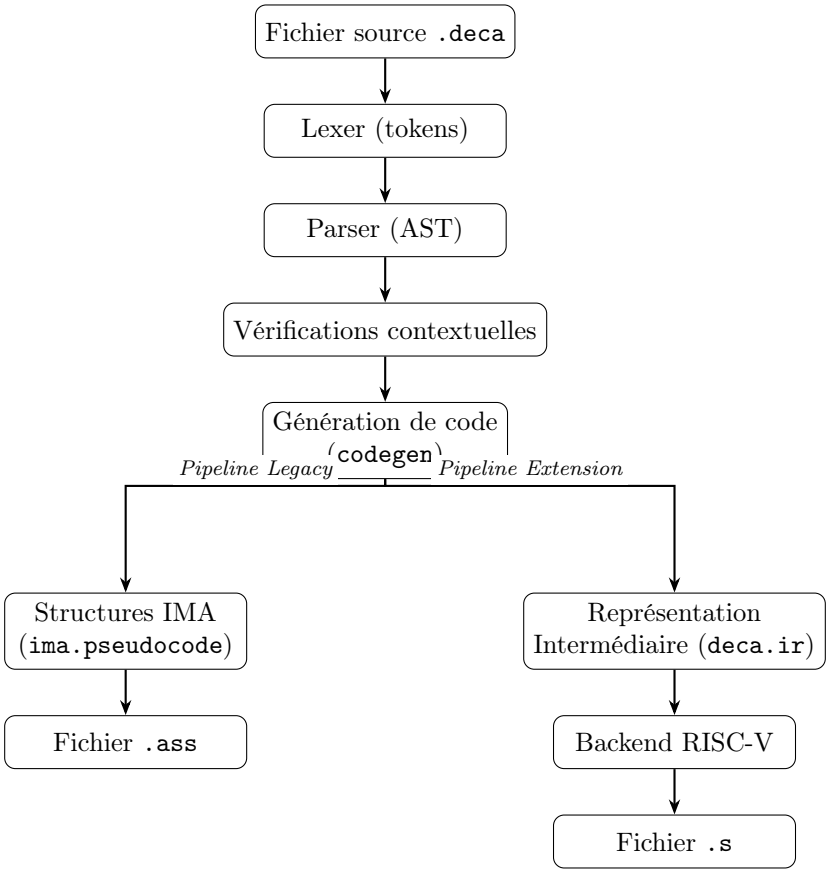


FIGURE 2 – Flux de compilation hybride : Pipeline direct (IMA) et modulaire (IR)

11 Conclusion

Le compilateur Deca est organisé selon une architecture modulaire claire, séparant les différentes phases de compilation (Phase A : analyse lexicale/syntaxique, Phase B : vérifications contextuelles, Phase C : génération de code) en packages distincts. L'introduction d'une représentation intermédiaire (IR) permet de découpler le frontend du backend, facilitant l'ajout de nouvelles cibles (comme RISC-V).

Cette organisation facilite la maintenance, les tests et l'évolution du projet. La documentation Javadoc et les tests automatisés garantissent la qualité et la maintenabilité du code.